DESPLIEGUE DE APLICACIONES WEB · 2º DAW

Servidores web y despliegue

Bloque 2.2 · Servidores web, servidores de aplicaciones y despliegue



Hoy entramos en la cocina

Nuevos personajes en el restaurante

El comensal, el camarero y la comanda siguen ahí. Hoy conocemos el resto del local.



Pan y bebidas

= Contenido estático

Ya está preparado



Plato al momento

= Contenido dinámico

Se cocina en cada pedido



La barra

= Servidor web

Sirve rápido y pasa comandas



Los cocineros

= Servidor de aplicaciones

Ejecutan las recetas



Locales satélite

= CDN

Copias cerca de cada barrio



Jefe de sala

= Balanceador de carga

Reparte comandas entre cocinas



Lote envasado

= Artefacto

Sellado, etiquetado y probado



Abrir al público

= Desplegar

Que lleguen los clientes reales

Menú del día

APERITIVO	Contenido estático vs. dinámico	10'
ENTRANTE	Renderizado, CDN y SEO	25'
PLATO PRINCIPAL	Servidor web vs. servidor de aplicaciones	45'
POSTRE	Qué significa desplegar	30'
SOBREMESA	Todo junto: de la petición al despliegue	10'

APERITIVO

Contenido estático vs. dinámico

Lo que ya está preparado y lo que se cocina al momento



Estático y dinámico



Contenido estático

Ficheros que el servidor envía tal cual están en disco, sin ejecutar código.

Ejemplos: HTML fijo, CSS, JavaScript del cliente, imágenes, PDF, fuentes.

Muy rápido: solo leer el fichero y enviarlo.

EN EL RESTAURANTE

El pan y las bebidas: ya están listos, solo hay que llevarlos a la mesa.



Contenido dinámico

El servidor ejecuta código en cada petición para generar la respuesta.

Ejemplos: Resultados de búsqueda, precios actualizados, el feed de cada usuario.

Más lento, pero imprescindible cuando los datos cambian.

EN EL RESTAURANTE

El plato que se cocina al momento, a gusto de cada comensal.

Una misma página mezcla los dos

```
GET /tienda          → HTML dinámico (precios)
                       que a su vez referencia:
  /css/estilos.css   → estático
  /img/logo.png      → estático
  /js/analytics.js   → estático
```

Lo habitual: servir lo estático de la forma más directa posible y reservar el coste de «ejecutar código» para lo que de verdad cambia.



En la misma mesa conviven el plato del día, recién cocinado, y el pan, el agua y el aceite, que ya estaban listos. Nadie pone a un cocinero a «preparar» el agua.

ENTRANTE

Renderizado, CDN y SEO

¿Dónde se emplata el plato?



¿Dónde se monta el HTML? CSR y SSR



CSR

Client-Side Rendering

El servidor envía un HTML casi vacío y un paquete grande de JavaScript. El navegador lo ejecuta y construye la página.

Típico de las SPA (React, Vue, Angular por defecto).

Navegación fluida tras cargar · primera carga lenta y página en blanco hasta que termina el JS.

EN EL RESTAURANTE

Te traen los ingredientes y un kit: el plato se monta en tu mesa.



SSR

Server-Side Rendering

El servidor ejecuta la lógica y envía el HTML ya completo en cada petición.

PHP o JSP clásicos; hoy también Next.js (React) y Nuxt (Vue).

La página se ve de inmediato, aunque el JavaScript aún no haya cargado.

EN EL RESTAURANTE

El plato sale de la cocina ya emplatado.

Hidratación y SSG



Hidratación

El servidor manda el HTML ya renderizado (SSR) y después el navegador «engancha» el JavaScript a ese HTML para que vuelva a ser interactivo, sin redibujarlo.

Lo mejor de los dos mundos: carga inicial rápida (como SSR) e interactividad rica después (como CSR).

EN EL RESTAURANTE

Te sirven el plato emplatado y, después, te traen las salsas y los cubiertos para que lo disfrutes a tu manera.



SSG

Static Site Generation: el HTML se genera una sola vez, en tiempo de build, antes de desplegar. El resultado es contenido estático.

Ideal para lo que no cambia por usuario (blog, documentación). Si hay precios o sesión, no basta.

EN EL RESTAURANTE

El menú del día, preparado por la mañana antes de abrir: se sirve al instante, pero es igual para todos.

CSR, SSR y SSG, cara a cara

	CSR	SSR	SSG
¿Dónde se genera el HTML?	En el navegador	En el servidor, en cada petición	En el build, una sola vez
Primera carga	Lenta	Rápida	Muy rápida
¿Servidor de aplicaciones en producción?	No	Sí	No
SEO	Más difícil	Bueno	Muy bueno
Caso típico	Panel de administración, app interna	Tienda online, red social	Blog, documentación
En el restaurante	*Kit para montar en la mesa*	*Emplatado en cocina*	*Menú del día preparado antes de abrir*

La fila clave para este módulo: si hace falta o no un servidor de aplicaciones en producción.

SEO: el crítico gastronómico

Los crawlers de los buscadores

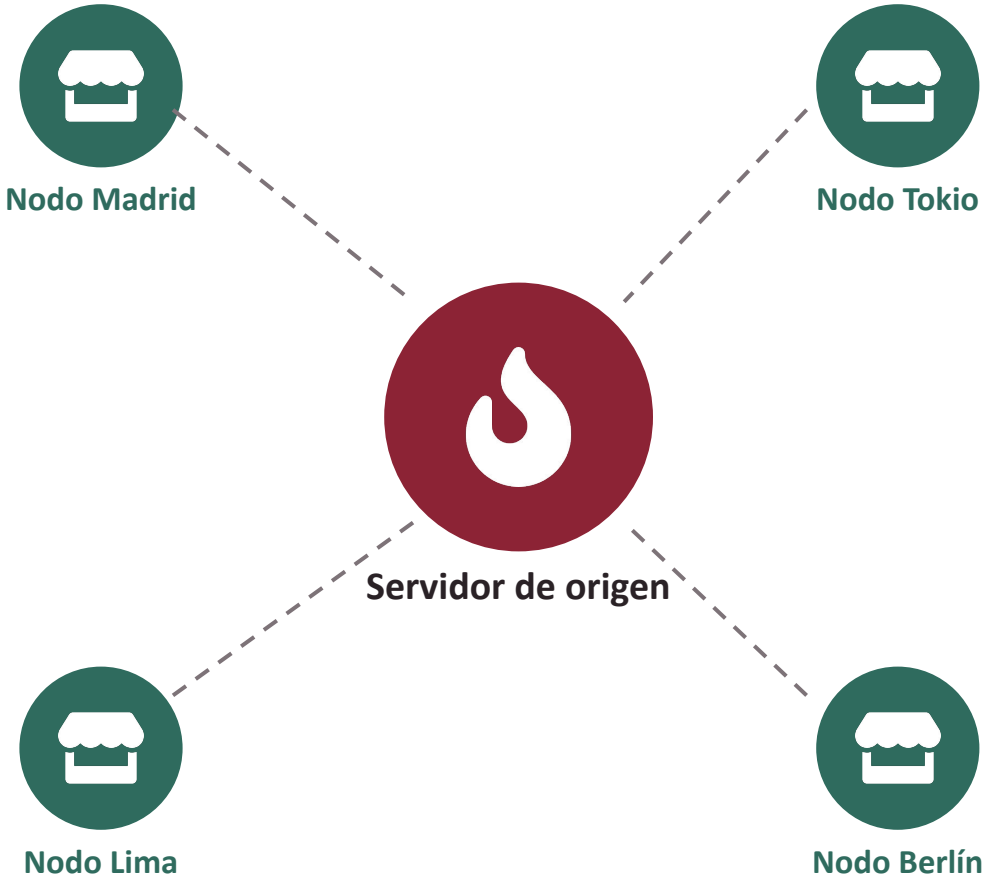
Visitan las páginas para indexarlas. Con CSR reciben al principio un HTML casi vacío y tienen que ejecutar el JavaScript para ver el contenido; no todos lo hacen igual de bien ni de rápido. SSR y SSG entregan el contenido ya visible.

Si al crítico le sirves un kit sin montar, no puede valorar el plato. Si se lo sirves emplatado, lo ve y lo puntúa al instante.

No es solo el crawler: también cuenta…

- <title> y <meta name="description"> bien escritos en cada página
- URLs legibles y estables; si cambian, redirigir con 301
- Tiempos de carga bajos: las páginas lentas se penalizan
- Contenido esencial (texto, enlaces) accesible sin JavaScript

CDN: locales satélite cerca de cada barrio



Red de servidores repartidos por el mundo que guardan copias cacheadas del contenido, cerca de cada usuario.

Cloudflare · Akamai · Fastly · Amazon CloudFront

Lo estático la aprovecha al máximo

CSS, JS, imágenes, HTML de SSG: igual para todos, fácil de cachear.

Lo personalizado, normalmente no

Tu carrito o tu sesión se siguen pidiendo al servidor de origen.

Los postres y el pan se reparten a locales satélite; los platos a medida se siguen cocinando en la central.

El problema de invalidar la caché



El problema

Despliegas un nuevo estilos.css, pero la CDN (o el propio navegador) sigue sirviendo la copia antigua. Los usuarios no ven el cambio.



La solución: cache busting

Incluir un hash o una versión en el nombre del fichero. Si el contenido cambia, cambia la URL, y la copia antigua deja de usarse sola.

```
estilos.css   →   estilos.a3f9c1.css   →   estilos.7be20d.css
```

Vite y webpack lo hacen automáticamente al hacer el build del frontend.



Si cambias la receta de la tarta, los locales seguirán sirviendo la vieja. La llamas «tarta de queso v2» y ya nadie puede pedir la antigua.

PLATO PRINCIPAL

Servidor web vs. servidor de aplicaciones

La barra y los cocineros



Servidor web: la barra



Recibe peticiones HTTP y responde

Su función principal es servir **contenido estático** con muchísima eficiencia.

También hace de «puerta de entrada» hacia otros servicios: **proxy inverso** (UD3).

Apache HTTP Server

El veterano: muy flexible, módulos para todo.

Nginx

Muy eficiente con miles de conexiones simultáneas.



La barra sirve al momento lo que ya está hecho (agua, pan, bebidas) y, cuando le piden un plato, no lo cocina: pasa la comanda a la cocina y devuelve el plato a la mesa.

Un truco de la barra: la compresión

Antes de enviar texto (HTML, CSS, JS, JSON), el servidor web puede comprimirlo con gzip o, más moderno, brotli.

Petición

```
GET /css/estilos.css HTTP/1.1
Host: www.miweb.com
Accept-Encoding: gzip, br
```

Respuesta

```
HTTP/1.1 200 OK
Content-Type: text/css
Content-Encoding: br
```

Reduce mucho lo que se transfiere sin tocar una línea de la aplicación: es configuración del servidor web, no del código.



Como la comida para llevar envasada al vacío: ocupa menos durante el viaje y se «desenvasa» al llegar a casa.

Servidor de aplicaciones: los cocineros



Ejecuta la lógica de la aplicación

Genera contenido dinámico y entiende el lenguaje o framework de la aplicación. Un servidor web puro no sabe «ejecutar» ese código.

Tomcat (Java) · Node.js · Gunicorn / uWSGI (Python) · PHP-FPM



Los cocineros conocen las recetas y cocinan cada plato cuando llega la comanda. No atienden la puerta ni sirven el agua.

	Servidor web	Servidor de aplicaciones
Función principal	Recibir peticiones y servir ficheros	Ejecutar la lógica de negocio
Tipo de contenido	Estático	Dinámico
Ejemplos	Apache, Nginx	Tomcat, Node.js, Gunicorn, PHP-FPM
¿Entiende el lenguaje de la app?	No	Sí
En el restaurante	*La barra*	*Los cocineros*

Cada cocina, su equipo

Lenguaje	Framework	Servidor de aplicaciones típico
Java	Spring (Spring Boot)	**Tomcat (a menudo embebido dentro del propio .jar)**
Python	Django, Flask	**Gunicorn o uWSGI**
PHP	Laravel, Symfony	**PHP-FPM**
Node.js	Express, NestJS	**El propio proceso de Node.js**
Ruby	Ruby on Rails	**Puma o Unicorn**



Una pizzería necesita horno de leña; un japonés, su barra de sushi. Cada framework necesita el servidor que sabe «cocinar» su lenguaje.

Spring Boot lleva Tomcat dentro del .jar: se arranca con java -jar app.jar y ya escucha peticiones.

¿Y Vue, Angular o React? Son frontend

Su código se ejecuta en el navegador (CSR), no en un servidor de aplicaciones. No sustituyen a Spring, Django o Laravel: conviven con ellos a través de una API.



Una SPA en CSR no necesita Node.js en producción. Node.js solo hizo falta para el build.

Excepción: SSR. Con Next.js o Nuxt el HTML se genera en cada petición, así que sí hace falta un proceso de Node.js en producción.

CORS (1): ¿qué es un origen?

Origen = esquema + host + puerto. Dos URL son del mismo origen solo si coinciden las tres partes.

Comparado con https://miapp.com	¿Mismo origen?	Motivo
`https://miapp.com/api/productos`	**Sí**	Solo cambia la ruta
`http://miapp.com`	**No**	Cambia el esquema
`https://api.miapp.com`	**No**	Cambia el host
`https://miapp.com:8080`	**No**	Cambia el puerto



El origen es la dirección completa del restaurante, puerta incluida. La entrada por la terraza (otro puerto) ya cuenta como otro local.

Subdominio distinto = origen distinto, aunque sea «la misma empresa».

CORS (2): quién da el permiso

Política del mismo origen: el JavaScript de un origen no puede leer las respuestas de otro, salvo que ese otro lo permita con cabeceras CORS.

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://miapp.com
Content-Type: application/json
```

1. **Sin esa cabecera:** el navegador descarta la respuesta antes de entregarla al JavaScript, aunque la petición haya llegado bien al servidor.
2. **Peticiones «no simples»:** (PUT, DELETE, JSON con cabeceras propias): antes se envía un OPTIONS de comprobación, el preflight.
3. **Error típico:** funciona con curl o Postman, pero falla en el navegador. CORS lo aplica el navegador, no el servidor.



Tu mesa (miapp.com) pide un plato a la cocina de otro local (api.miapp.com). El camarero lo trae, pero solo te lo deja en la mesa si el otro local ha puesto en la etiqueta «se puede servir a miapp.com». Para pedidos especiales, antes se llama por teléfono para preguntar si los aceptan (preflight).

Cómo atienden por dentro: el modelo de concurrencia

Nginx

Eventos asíncronos: pocos procesos atienden miles de conexiones sin bloquearse.

Un camarero que toma nota en una mesa y pasa a la siguiente sin quedarse esperando.

Apache (prefork)

Un proceso o hilo por conexión. Sencillo, pero más pesado con mucha concurrencia (existe también el modo event).

Un camarero asignado a cada mesa durante toda la comida.

Node.js

Un único hilo con un event loop. Muy bueno con E/S (red, disco); malo si una petición hace cálculos pesados.

Un solo camarero rapidísimo que nunca espera… salvo que se ponga a cocinar algo largo: entonces se para todo.

Gunicorn / Tomcat

Varios workers en paralelo, cada uno con una petición a la vez. La concurrencia depende de cuántos se configuren.

N cocineros, cada uno con un plato a la vez.

Los dos a la vez: el patrón habitual en producción



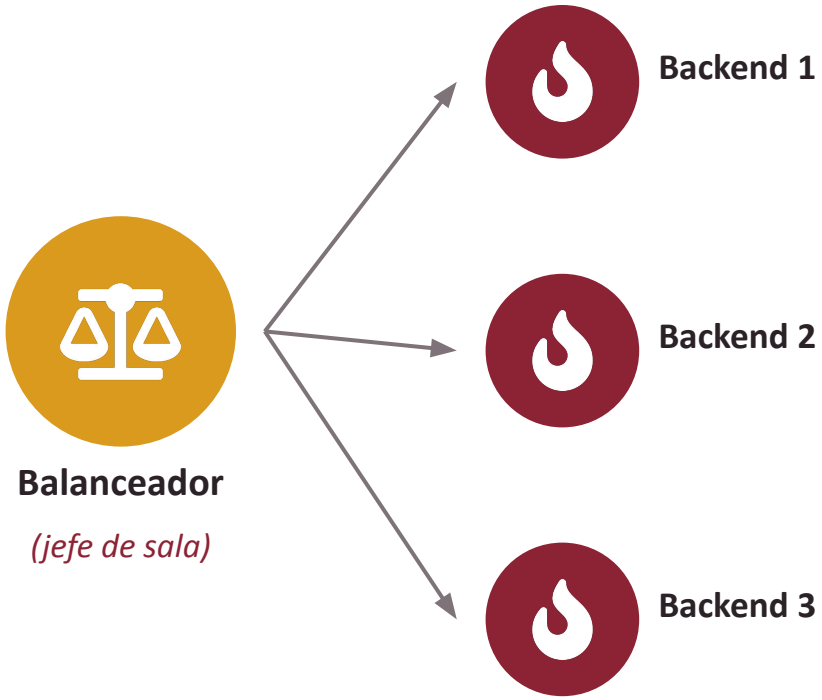
`/css/*, /img/*` → responde directamente (estático)

`/api/*` → reenvía (proxy inverso) a Node.js / Tomcat, que consulta la base de datos y genera la respuesta dinámica

502 Bad Gateway: la app no está arrancada o responde mal. **504 Gateway Timeout:** la app tarda demasiado.

La barra sirve el pan directamente y pasa las comandas de platos a la cocina.

Varias cocinas: el balanceo de carga



Estrategia	Idea	En el restaurante
Round robin	Por turnos, uno tras otro	*Una comanda a cada cocina, por orden*
Weighted round robin	Por turnos, pero los más potentes reciben más	*La cocina grande recibe el doble*
Least connections	Al que tiene menos conexiones activas	*A la cocina menos ocupada*
IP hash	El mismo cliente, siempre al mismo backend	*La mesa 7 siempre a la misma cocina*

«Servidor de aplicaciones» no tiene por qué significar «una única instancia».

Escalado vertical vs. horizontal

Vertical



Más recursos (CPU, RAM) para la misma máquina.

Tiene un límite físico y suele obligar a parar el servicio para ampliar.

Reformar la cocina para meter más fogones: hay que cerrar, y el local no crece para siempre.

Horizontal



Más instancias en paralelo, con un balanceador delante.

El enfoque habitual en la nube: no tiene un límite tan rígido.

Abrir más cocinas iguales y repartir las comandas.

Docker facilita el escalado horizontal: una instancia más es, en esencia, un contenedor más de la misma imagen.

¿Sigue vivo? Health checks y orquestadores



Health checks

El balanceador comprueba periódicamente que cada backend responde bien, con una petición de prueba a una ruta tipo /health.

Si un backend deja de responder, se retira de la rotación hasta que vuelve a estar sano: los usuarios no reciben errores.

EN EL RESTAURANTE

El jefe de sala asoma la cabeza a cada cocina: «¿seguís operativos?». Si una no contesta, deja de mandarle comandas.



Orquestadores

Arrancar, vigilar y balancear a mano docenas de contenedores no escala. Un orquestador (el más conocido, Kubernetes) lo automatiza.

Decide cuántas instancias levantar, las reinicia si fallan, reparte el tráfico y facilita los despliegues graduales. No lo usaremos en el módulo.

EN EL RESTAURANTE

El gerente de la cadena: abre y cierra cocinas según la demanda y sustituye las que se averían.

¿Por qué no servirlo todo desde la aplicación?

Técnicamente se puede. Pero no se pone al chef a servir el agua.



Eficiencia

Los servidores web están optimizados para servir estáticos y aguantar miles de conexiones simultáneas.



Caché

Lo estático se cachea con facilidad en el servidor web o en una CDN.



HTTPS en un solo punto

El cifrado se configura una vez, en la puerta de entrada, y no en cada aplicación.



Balanceo

La misma puerta de entrada reparte la carga entre varios servidores de aplicaciones.

Separar responsabilidades: la misma idea que las capas del bloque 2.1, aplicada a la infraestructura.

POSTRE

Qué significa «desplegar»

Abrir el restaurante al público



Desplegar, en una frase

Poner una aplicación a disposición de sus usuarios reales, ejecutándose en algún sitio accesible desde fuera de tu máquina de desarrollo.



Dominio

El nombre que escriben los usuarios: miapp.com

El nombre del restaurante



DNS

Apunta el dominio a la IP del servidor

La guía de restaurantes



Hosting

Dónde se ejecuta físicamente la aplicación

El local



Certificado HTTPS

Para que la comunicación vaya cifrada (UD3)

Hablar en clave con el camarero



Artefacto

El paquete concreto y versionado que se despliega

El lote envasado y etiquetado

Tipos de hosting, de menos a más control

más control →

Compartido

Varias webs comparten un servidor gestionado por el proveedor. Barato, poco control.

Cocina compartida por horas con otros negocios.

VPS

Servidor privado virtual: una máquina virtual para ti solo, con control total del sistema operativo.

Tu propio local dentro de un centro comercial.

Dedicado

Una máquina física entera para ti.

Un edificio entero para tu restaurante.

Cloud

Recursos bajo demanda, escalables y facturados por uso: desde máquinas virtuales (IaaS) hasta plataformas donde solo subes código o contenedor (PaaS).

Fogones de alquiler que crecen o se reducen según los clientes.

Entornos: no hay un único «producción»



Desarrollo

Tu máquina: programar y probar en local.

Tu cocina de casa



Staging

Réplica de producción para probar antes de publicar.

El ensayo general antes de abrir



Producción

El entorno real, con usuarios de verdad.

El servicio con clientes



El mismo artefacto versionado se promociona de un entorno a otro sin reconstruirlo. Lo probado en staging es exactamente lo que llega a producción.



El lote que se probó en el ensayo general es el que se sirve en la apertura: no se vuelve a cocinar.

¿Qué es exactamente un artefacto?

Artefacto: el resultado empaquetado de construir la aplicación. Es lo que de verdad se copia, instala o ejecuta en el servidor, no el código fuente suelto.



Lenguajes compilados

El binario o paquete compilado.

`.jar / .war (Java) · .exe · binario de Go`



Lenguajes interpretados

También existe: un paquete con el código listo para ejecutar junto con sus dependencias.

`paquete npm · wheel .whl (Python) · .tar.gz (PHP)`



Imagen Docker

Empaqueta el artefacto junto con todo su entorno de ejecución. Es un artefacto versionado por su tag.

`miapp:1.4.0`



Un lote envasado, sellado y con su número: se prueba, y ese mismo lote es el que llega a todos los locales.

El caso de PHP: un lenguaje que no se compila

Lo que NO se debe hacer

```
$ git pull          # en producción
$ composer install # cada vez
```

Una reinstalación puede traer una versión distinta de una dependencia. Lo que llega a producción no es lo que se probó.

EN EL RESTAURANTE

Ir al mercado cada mañana: un día el proveedor te da otra marca y el plato ya no sabe igual.

El artefacto

```
miapp-1.4.0.tar.gz
  ├── src/   (código)
  └── vendor/   (dependencias resueltas)
```

Se genera una vez, es inmutable, se prueba y se despliega exactamente ese paquete.

EN EL RESTAURANTE

Un kit con todos los ingredientes ya comprados y sellados.

Repositorios de artefactos y versiones



Repositorios de artefactos

Docker Hub · JFrog Artifactory · Sonatype Nexus · npm · PyPI

Se sube una vez cada versión y siempre se descarga la misma.

Son la «fuente de verdad»: ante un incidente se comprueba qué versión hay desplegada y se descarga para reproducir el problema.

El almacén central de lotes, cada uno con su número.

`SemVer: MAJOR.MINOR.PATCH`

Parte	Cuándo sube	Ejemplo	En la carta
`MAJOR`	Cambios incompatibles	2.4.1 → 3.0.0	*Carta nueva entera*
`MINOR`	Funcionalidad nueva compatible	2.4.1 → 2.5.0	*Un plato nuevo*
`PATCH`	Corrección de errores	2.4.1 → 2.4.2	*Corregir la sal*

Nada de «la última» (latest). Versionar explícitamente permite saber qué hay en cada entorno, repetir un despliegue idéntico y hacer rollback a una versión conocida.

Configuración y secretos: lo único que cambia

Si el artefacto es el mismo en todos los entornos, ¿cómo sabe a qué base de datos conectarse en cada uno?

```
# Mismo artefacto, distinta configuración según el entorno
docker run -e DB_HOST=db-staging -e DB_PASSWORD=*** miapp:1.4.0   # staging
docker run -e DB_HOST=db-prod    -e DB_PASSWORD=*** miapp:1.4.0   # producción
```

- Variables de entorno
- Ficheros de configuración montados aparte
- Gestor de secretos (Vault, AWS Secrets Manager)

Contraseñas, tokens y claves nunca en el código ni «horneadas» dentro de la imagen.

La receta es la misma en todos los locales. Cambian el proveedor de cada barrio y la combinación de la caja fuerte, que nunca se escribe en la receta.

Un flujo de despliegue simple (sin CI/CD todavía)

1. **Código y Dockerfile en Git**
2. **Construir el artefacto versionado**
3. **Subirlo a un registry**
4. **Descargar esa versión en el servidor y ejecutarla**
5. **Nginx delante enruta el tráfico**

```
# En tu máquina (o en el pipeline)
docker build -t usuario/miapp:1.4.0 .
docker push usuario/miapp:1.4.0

# En el servidor
docker pull usuario/miapp:1.4.0
docker run -d --name miapp -p 3000:3000 \
    usuario/miapp:1.4.0
```

El contenedor es una cocina dentro de un contenedor de verdad: la que probaste en tu portátil es, en esencia, la que se lleva tal cual a producción.

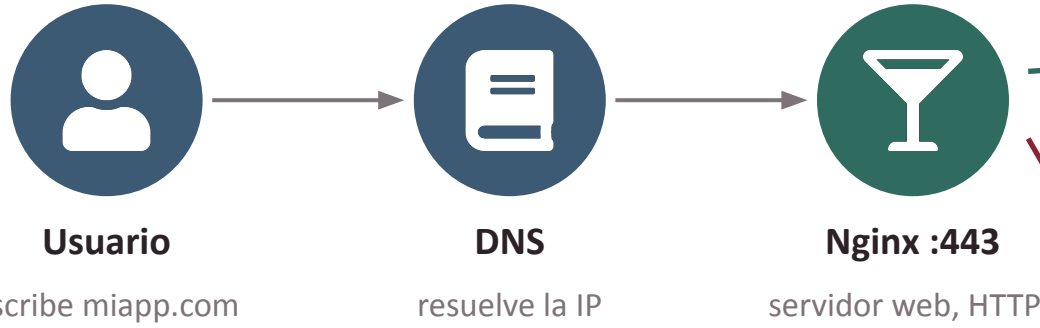
SOBREMESA

Todo junto

De la comanda al despliegue



El panorama completo



`/css/estilo.css` → Nginx o una CDN (estático)

`/api/productos` → uno de los backends sanos (health check), que ejecuta miapp:1.4.0, consulta la BD y devuelve JSON

Cada backend ejecuta una versión concreta de un artefacto, descargada de un repositorio.

Varios backends idénticos, balanceados y vigilados con health checks.

La configuración y los secretos llegan desde fuera del artefacto.

Desplegar una versión nueva sin cortar el servicio



Rolling (progresivo)

Se sustituyen los backends antiguos por los nuevos de uno en uno, mientras los demás siguen atendiendo.

EN EL RESTAURANTE

Renovar la cocina fogón a fogón, sin cerrar el restaurante.



Blue-green

Se levanta en paralelo un entorno completo con la versión nueva (green). Verificado, se pasa todo el tráfico; el antiguo (blue) queda para un rollback inmediato.

EN EL RESTAURANTE

Montar un comedor nuevo al lado y cambiar a los clientes de sala de golpe.



Canary

La versión nueva recibe primero un pequeño porcentaje del tráfico; si va bien, se amplía poco a poco.

EN EL RESTAURANTE

Probar el plato nuevo solo en unas pocas mesas.

Con usuarios reales no se puede «apagar y encender». Se profundiza en la UD5.

Lanzar, migrar y vigilar



Feature flags

Activan o desactivan una funcionalidad sin volver a desplegar. Separan «desplegar» de «lanzar»: el código llega apagado y se enciende cuando se decide. Si falla, se apaga: rollback casi instantáneo.

EN EL RESTAURANTE

El plato ya se sabe cocinar, pero no está en la carta hasta que lo decides.



Migraciones de BD

Si la versión nueva cambia el esquema, volver al artefacto anterior no deshace ese cambio. Por eso las migraciones se diseñan compatibles hacia atrás (añadir una columna opcional en vez de renombrarla).

EN EL RESTAURANTE

Si reorganizas la despensa, volver a la receta antigua no devuelve cada ingrediente a su sitio.



Monitorización y rollback

Tras desplegar, vigilar los errores 5xx y el tiempo de respuesta. Si falla, volver a desplegar la versión anterior conocida, en lugar de arreglarlo en caliente.

EN EL RESTAURANTE

Mirar las quejas y los tiempos de espera; si el plato nuevo falla, volver a la receta de ayer.

No es una caja negra: qué viene después

Pieza	Dónde se ve
Docker, imágenes, registries	**UD1**
DNS, HTTP, arquitecturas	**UD2 · bloque 2.1**
Apache y Nginx a fondo: hosts virtuales, HTTPS, proxy inverso, rendimiento	**UD3**
Servidores de aplicaciones (Tomcat y similares) y transferencia de ficheros	**UD4**
Automatización del despliegue (CI/CD)	**UD5**



Cada pieza del restaurante que hemos visto hoy tiene su unidad: ya conocéis el local; ahora toca aprender a montarlo.

La cuenta, por favor: ideas clave



Estático = fichero servido tal cual; dinámico = código en cada petición. Una página mezcla los dos.



CSR monta el HTML en el navegador, SSR en el servidor y SSG en el build. SSR y SSG ayudan al SEO.



Servidor web (Nginx, Apache): estático, compresión y proxy. Servidor de aplicaciones: ejecuta la lógica.



CORS: el backend autoriza orígenes (esquema + host + puerto); lo aplica el navegador.



Producción: Nginx delante y varios backends detrás, con balanceo y health checks.



Se despliega un artefacto inmutable y versionado, el mismo en todos los entornos, con la configuración fuera.

PARA LLEVAR

El restaurante, abierto



A continuación: práctica del bloque 2.2.

Próxima unidad: UD3, Apache y Nginx a fondo.